

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЁТ
по лабораторной работе №1
по дисциплине «Базы данных»
Тема: Выборка данных и модификация данных.
Команды SELECT, INSERT, DELETE, UPDATE.

Студент гр. 4352	_____	Гиршович Д.И.
Студентка гр. 4352	_____	Слабнова Д.А.
Студент гр. 4352	_____	Даричев Е.М.
Преподаватель	_____	Преподаватель А.А.

Санкт-Петербург

2026

Цель работы.

Изучение основных SQL-команд для выборки, обработки и модификации данных в PostgreSQL.

Ход выполнения работы.

Для выполнения запросов использовалась консольная утилита `psql`:

```
psql -U postgres demo
```

Упражнение 1 — Извлечение данных.

1. Изучите таблицу *seats*.

Для просмотра структуры таблицы использовалась специальная команда PostgreSQL:

```
\d seats
```

```
Table "bookings.seats"
  Column      |      Type      | Nullable
-----+-----+-----
airplane_code | character(3)    | not null
seat_no       | text           | not null
fare_conditions | text           | not null
Indexes: "seats_pkey", btree (airplane_code, seat_no)
```

Листинг 1 — Результат выполнения запроса.

Таблица содержит код самолёта, номер места и класс обслуживания. Пара полей `airplane_code` и `seat_no` входит в первичный ключ, поэтому такая комбинация уникальна.

2. Извлеките все данные из таблицы *seats*.

Для извлечения всех столбцов и строк таблицы используется оператор `SELECT *`. Звёздочка обозначает выбор всех столбцов без перечисления их имён.

```
SELECT *
FROM seats;
```

airplane_code	seat_no	fare_conditions
32N	10A	Economy
32N	10B	Economy
32N	10C	Economy
...		

(1741 rows)

Листинг 2 — Результат выполнения запроса.

Команда `SELECT * FROM seats` извлекает все столбцы и все строки указанной таблицы. В результате получено 1741 строка.

3. Модифицируйте предыдущий запрос с использованием оператора `ORDER BY`.

```
SELECT *
FROM seats
ORDER BY airplane_code ASC, seat_no DESC;
```

airplane_code	seat_no	fare_conditions
32N	9F	Economy
32N	9E	Economy
32N	9D	Economy
...		
E70	10C	Economy
E70	10B	Economy
E70	10A	Economy

(1741 rows)

Листинг 3 — Результат выполнения запроса.

Оператор `ORDER BY` задаёт порядок сортировки результата. Параметр `ASC` сортирует коды самолётов по возрастанию, а `DESC` – номера мест по убыванию внутри каждого кода. Таким образом, сначала коды самолётов расположены в алфавитном порядке, а места каждого самолёта – в обратном алфавитном порядке.

4. Получите количество строк в таблице `seats` с использованием оператора `COUNT(*)`.

```
SELECT COUNT(*)
FROM seats;
```

```

count
-----
1741
(1 row)

```

Листинг 4 — Результат выполнения запроса.

Функция COUNT(*) является агрегатной функцией и подсчитывает количество строк таблицы. Полученное значение 1741 совпадает с количеством строк из пункта 2.

- Получите количество сидений в самолёте с airplane_code, равным 32N, с использованием оператора WHERE.

Оператор WHERE ограничивает выборку строками, удовлетворяющими условию. В данном случае подсчитываются только места самолёта с кодом 32N.

```

SELECT COUNT(*)
FROM seats
WHERE airplane_code = '32N';

```

```

count
-----
166
(1 row)

```

Листинг 5 — Результат выполнения запроса.

- Получите airplane_code, в которых присутствуют seat_no, равные 30C или 51G, с использованием оператора IN.

Оператор IN проверяет принадлежность значения столбца заданному набору значений. Он заменяет несколько условий, объединённых оператором OR, и позволяет найти самолёты, в которых есть места 30C или 51G.

```

SELECT airplane_code
FROM seats
WHERE seat_no IN ('30C', '51G');

```

```

airplane_code
-----
32N
339
339
351
77W
77W
789
(7 rows)

```

Листинг 6 — Результат выполнения запроса.

В выводе встречаются повторяющиеся коды, так как один самолёт может иметь оба указанных места. Уникальные коды: 32N, 339, 351, 77W, 789.

Упражнение 2 — Модификация отображения извлечённых данных.

1. Получите набор уникальных кодов самолётов из таблицы *seats* с использованием опции `DISTINCT`.

```

SELECT DISTINCT airplane_code
FROM seats
ORDER BY airplane_code;

```

```

airplane_code
-----
32N
339
351
77W
789
7M7
CR7
E70
(8 rows)

```

Листинг 7 — Результат выполнения запроса.

Ключевое слово `DISTINCT` удаляет повторяющиеся значения из результата запроса. В данном случае оно позволяет получить список кодов самолётов без повторений.

2. Добавьте к списку выборки поле `airplane_seats`, в котором выводятся `airplane_code` и `seat_no`, объединённые через пробел.

```
SELECT *,
        airplane_code || ' ' || seat_no AS airplane_seats
FROM seats;

airplane_code | seat_no | fare_conditions | airplane_seats
-----+-----+-----+-----
32N           | 10A     | Economy         | 32N 10A
32N           | 10B     | Economy         | 32N 10B
...
(1741 rows)
```

Листинг 8 — Результат выполнения запроса.

Оператор `||` в PostgreSQL объединяет несколько строковых значений в одну строку. В данном запросе код самолёта и номер места соединяются через пробел, а ключевое слово `AS` задаёт псевдоним вычисляемого столбца `airplane_seats`.

3. Разбейте значение из поля `seat_no` на два столбца: целочисленный номер и букву.

```
SELECT
    seat_no,
    CASE
        WHEN LENGTH(seat_no) = 2
            THEN SUBSTRING(seat_no FROM 1 FOR 1)
            ELSE SUBSTRING(seat_no FROM 1 FOR 2)
    END AS seat_number,
    CASE
        WHEN LENGTH(seat_no) = 2
            THEN SUBSTRING(seat_no FROM 2 FOR 1)
            ELSE SUBSTRING(seat_no FROM 3 FOR 1)
    END AS seat_letter
FROM seats;
```

seat_no	seat_number	seat_letter
10A	10	A
10B	10	B
...		

Листинг 9 — Результат выполнения запроса.

Конструкция CASE ... END является условным выражением SQL: WHEN задаёт условие, THEN – результат при его выполнении, а ELSE – результат в остальных случаях. В сочетании с функциями LENGTH и SUBSTRING конструкция учитывает, состоит ли номер места из двух или трёх символов, и разделяет его на числовую часть и букву.

Упражнение 3 — Агрегация данных.

1. Получите количество сидений для каждого кода самолёта с использованием оператора GROUP BY и агрегатной функции COUNT(*).

Оператор GROUP BY объединяет строки с одинаковым значением airplane_code в группы. Агрегатная функция COUNT(*) подсчитывает количество мест в каждой группе.

```
SELECT airplane_code, COUNT(*) AS seats_count
FROM seats
GROUP BY airplane_code
ORDER BY airplane_code;
```

airplane_code	seats_count
32N	166
339	281
351	325
77W	404
789	257
7M7	160
CR7	70
E70	78

(8 rows)

Листинг 10 — Результат выполнения запроса.

2. Модифицируйте предыдущий запрос с использованием оператора HAVING, чтобы получить только самолёты с количеством мест более 160.

```
SELECT airplane_code, COUNT(*) AS seats_count
FROM seats
GROUP BY airplane_code
HAVING COUNT(*) > 160;
```

airplane_code	seats_count
32N	166
339	281
351	325
77W	404
789	257

(5 rows)

Листинг 11 — Результат выполнения запроса.

Оператор HAVING используется для фильтрации результатов, полученных после группировки с помощью GROUP BY. В отличие от WHERE, который отбирает отдельные строки до группировки, HAVING задаёт условие для уже сформированных групп. Поэтому в результат попали только самолёты, имеющие более 160 мест.

3. Объедините результаты двух запросов с использованием оператора UNION.

```
SELECT airplane_code, COUNT(*) AS seats_count, 'more than 160' AS
comment
FROM seats
GROUP BY airplane_code
HAVING COUNT(*) > 160
UNION
SELECT airplane_code, COUNT(*) AS seats_count, '160 or less' AS comment
FROM seats
GROUP BY airplane_code
HAVING COUNT(*) <= 160;
```

airplane_code	seats_count	comment
32N	166	more than 160
339	281	more than 160
351	325	more than 160
77W	404	more than 160
789	257	more than 160
7M7	160	160 or less
CR7	70	160 or less
E70	78	160 or less

(8 rows)

Листинг 12 — Результат выполнения запроса.

Оператор UNION объединяет результаты нескольких запросов в одну результирующую таблицу. Объединяемые запросы должны возвращать одинаковое количество столбцов с совместимыми типами данных. Оба запроса возвращают код самолёта, количество мест и текстовый комментарий, поэтому их результаты можно объединить.

- Получите общее количество мест в каждом самолёте и во всех самолётах суммарно с использованием конструкции ROLLUP.

```
SELECT airplane_code, COUNT(*) AS seats_count
FROM seats
GROUP BY ROLLUP (airplane_code)
ORDER BY GROUPING(airplane_code), airplane_code;
```

airplane_code	seats_count
32N	166
339	281
351	325
77W	404
789	257
7M7	160
CR7	70
E70	78
	1741

(9 rows)

Листинг 13 — Результат выполнения запроса.

Конструкция ROLLUP является расширением GROUP BY и позволяет вместе с результатами группировки получить итоговые значения. В данном случае помимо восьми строк с количеством мест для каждого самолёта сформирована девятая строка с общим количеством мест во всех самолётах. Поэтому количество строк результата равно 9, а итоговое количество мест равно 1741.

5. Замените в строке с общим числом мест значение NULL на ALL с использованием оператора GROUPING и конструкции CASE ... END.

```

SELECT CASE
    WHEN GROUPING(airplane_code) = 1 THEN 'ALL'
    ELSE airplane_code
END AS airplane_code,
COUNT(*) AS seats_count
FROM seats
GROUP BY ROLLUP (airplane_code)
ORDER BY GROUPING(airplane_code), airplane_code;

```

airplane_code	seats_count
32N	166
339	281
351	325
77W	404
789	257
7M7	160
CR7	70
E70	78
ALL	1741

(9 rows)

Листинг 14 — Результат выполнения запроса.

Функция GROUPING определяет, является ли строка обычным результатом группировки или итоговой строкой, созданной ROLLUP. В итоговой строке код самолёта имеет значение NULL, поэтому с помощью CASE оно заменяется на более наглядное значение ALL.

Упражнение 4 — Модификация данных в таблицах.

1. Удалите из таблицы *seats* все записи, относящиеся к самолёту 32N, с использованием команды DELETE.

```
DELETE FROM seats
WHERE airplane_code = '32N';
```

DELETE 166

Листинг 15 — Результат выполнения запроса.

Команда DELETE удаляет записи из таблицы. Оператор WHERE ограничивает удаление нужными строками; если не указать WHERE, из таблицы будут удалены все записи. После удаления в таблице осталось 1575 строк, записей с кодом 32N нет.

2. С использованием команды UPDATE измените значение поля *airplane_code* на 32N для записей со значением 339.

```
UPDATE seats
SET airplane_code = '32N'
WHERE airplane_code = '339';
```

Листинг 16 — Результат выполнения запроса.

Команда UPDATE изменяет существующие записи. После ключевого слова SET указывается новое значение поля, а оператор WHERE определяет строки, к которым применяется изменение. После обновления стало 281 запись с кодом 32N и ни одной записи с кодом 339.

3. С использованием команды INSERT добавьте три записи в таблицу *seats*.

```
INSERT INTO seats (airplane_code, seat_no, fare_conditions)
VALUES
  ('339', '1A', 'Business'),
  ('339', '1B', 'Business'),
  ('339', '1C', 'Business');
```

```
INSERT 0 3
```

airplane_code	seat_no	fare_conditions
339	1A	Business
339	1B	Business
339	1C	Business

Листинг 17 — Результат выполнения запроса.

Команда INSERT INTO добавляет новые записи в таблицу. После имени таблицы перечисляются заполняемые столбцы, а после ключевого слова VALUES – добавляемые значения. Одной командой INSERT можно добавить сразу несколько строк. После вставки количество строк стало равно 1578. Команды этого упражнения проверялись в транзакции и завершались ROLLBACK, поэтому исходное состояние базы сохранено.

Упражнение 5 — Запросы к нескольким таблицам.

1. Изучите таблицы *routes* и *airplanes_data*, содержащие данные о маршрутах и самолётах.

```
\d routes
```

```
\d airplanes_data
```

Table "bookings.routes"

Column	Type	Nullable
route_no	text	not null
validity	tstzrange	not null
departure_airport	character(3)	not null
arrival_airport	character(3)	not null
airplane_code	character(3)	not null
days_of_week	integer[]	not null
scheduled_time	time without time zone	not null
duration	interval	not null

Foreign-key constraints:

```
routes_airplane_code_fkey FOREIGN KEY (airplane_code)
REFERENCES airplanes_data(airplane_code)
```

Листинг 18 — Результат выполнения запроса.

Table "bookings.airplanes_data"

Column	Type	Nullable
airplane_code	character(3)	not null
model	jsonb	not null
range	integer	not null
speed	integer	not null

Indexes:

```
"airplanes_data_pkey" btree (airplane_code)
```

Листинг 19 — Результат выполнения запроса.

Таблица `routes` содержит данные о маршрутах, а `airplanes_data` – данные о самолётах. Для дальнейших запросов используются поля `airplane_code`, `days_of_week` и `scheduled_time` из `routes`. В таблице `airplanes_data` код самолёта является первичным ключом, а `routes.airplane_code` ссылается на него по внешнему ключу.

- Получите количество записей, объединив таблицы с использованием `INNER JOIN`, `LEFT JOIN` и `RIGHT JOIN`.

```
SELECT COUNT(*)
FROM routes
```

```
INNER JOIN airplanes_data
ON routes.airplane_code = airplanes_data.airplane_code;
```

```
count
-----
1162
(1 row)
```

Листинг 20 — Результат выполнения запроса.

```
SELECT COUNT(*)
FROM routes
LEFT JOIN airplanes_data
ON routes.airplane_code = airplanes_data.airplane_code;
```

```
count
-----
1162
(1 row)
```

Листинг 21 — Результат выполнения запроса.

```
SELECT COUNT(*)
FROM routes
RIGHT JOIN airplanes_data
ON routes.airplane_code = airplanes_data.airplane_code;
```

```
count
-----
1164
(1 row)
```

Листинг 22 — Результат выполнения запроса.

Оператор JOIN объединяет данные нескольких таблиц по заданному условию. INNER JOIN оставляет только строки, для которых найдено соответствие в обеих таблицах. LEFT JOIN сохраняет все строки левой таблицы routes, а RIGHT JOIN – все строки правой таблицы airplanes_data, даже если соответствующая запись в другой таблице отсутствует. Поэтому две дополнительные строки справа не имеют соответствия в routes.

3. Удалите из таблицы *seats* записи с *airplane_code*, равным 32N, и повторно выполните соединения.

```
DELETE FROM seats
WHERE airplane_code = '32N';
```

Команда DELETE изменяет данные только в таблице, указанной после DELETE FROM. После удаления было повторно выполнено соединение из пункта 2. Результат остался прежним: INNER JOIN – 1162, LEFT JOIN – 1162, RIGHT JOIN – 1164. Таблица *seats* не участвует в этих соединениях, поэтому её изменение не влияет на результат.

4. С использованием оператора BETWEEN получите количество мест для рейсов с временем вылета между 19:00:00 и 21:00:00 и *airplane_code* = 'E70'.

```
SELECT COUNT(*)
FROM routes
JOIN seats
  ON routes.airplane_code = seats.airplane_code
WHERE routes.scheduled_time BETWEEN '19:00:00' AND '21:00:00'
      AND routes.airplane_code = 'E70';
```

```
count
-----
1014
(1 row)
```

Листинг 23 — Результат выполнения запроса.

Оператор BETWEEN проверяет принадлежность значения заданному диапазону, включая обе его граничные значения. Поэтому в подсчёт попали рейсы E70 со временем вылета от 19:00:00 до 21:00:00 включительно.

5. С использованием функции ARRAY_POSITION и оператора IS NOT NULL получите самое позднее время вылета по понедельникам.

```
SELECT MAX(scheduled_time) AS latest_departure
FROM routes
WHERE ARRAY_POSITION(days_of_week, 1) IS NOT NULL;
```

```
latest_departure
-----
23:55:00
(1 row)
```

Листинг 24 — Результат выполнения запроса.

Функция `ARRAY_POSITION` возвращает позицию заданного значения внутри массива. Если искомое значение отсутствует, функция возвращает `NULL`, поэтому условие `IS NOT NULL` оставляет только строки, в которых найдено число 1. В массиве `days_of_week` число 1 обозначает понедельник, а агрегатная функция `MAX` выбирает самое позднее время вылета.

Выводы.

В ходе лабораторной работы изучены команды `SELECT`, `INSERT`, `UPDATE` и `DELETE`, фильтрация и сортировка данных с помощью `WHERE` и `ORDER BY`, агрегатные функции, группировка, `HAVING`, `UNION` и `ROLLUP`. Также выполнены запросы к нескольким таблицам с использованием `INNER JOIN`, `LEFT JOIN` и `RIGHT JOIN`. Освоены строковые функции, условное выражение `CASE`, функция `GROUPING`, работа с массивами и проверка результата через `ARRAY_POSITION`. Полученные результаты подтверждают, что выборка и модификация данных выполняются в PostgreSQL предсказуемо в соответствии с условиями запросов.